

Part C follow-up – match the requirement slide

The three warnings and the confirmation are in, but the wording differs from the slide, and the relearn warning behaves differently: it currently shows always, where the slide shows it only when that indicator actually has a deviation. Eight steps.

The slide shows	What is built now	Fixed by
/!\ Warning : Deviations are set on this A/C on Update Aircraft	A longer sentence about closing them	Step 1
The same text on Import configurations	The same longer sentence	Step 1
/!\ Warning : a deviation is set on this indicator on Threshold learning	A general note, shown on every relearn	Steps 2 to 7
A popup with the AirbusWorld TE instructions and Cancel / Continue buttons	A plain Yes / No confirmation	Step 8

The wording

Step 1 — CoreMessages.properties – and every language file

mgs-client-common/src/com/eurocopter/mgs/core/presentation/i18n/CoreMessages.properties

Replace the three warning texts with the slide's wording, and add the three new keys the steps below use. Each on one line, no backslash.

```
deviationConfigUpdateWarning=/!\ Warning : Deviations are set on this A/C
deviationConfigImportWarning=/!\ Warning : Deviations are set on this A/C
deviationRelearnIndicatorWarning=/!\ Warning : a deviation is set on this indicator
```

```
deviationConfigUpdateConfirmTitle=Condition Indicator Update
deviationConfigUpdateConfirm=Condition Indicators in the deviation list will be acknowledged\n\nPlease open a TE on\nAir
deviationContinue=Continue
```

In a properties file a single backslash escapes the next character, so the warning prefix is written as two backslashes to produce one. Likewise \n gives a line break in the popup text.

The key deviationRelearnNote is no longer used – delete it.

The relearn warning, per indicator

The slide warns about *this indicator*, so the warning must only appear when the threshold being relearned actually has a deviation. Three different places open the Threshold learning dialog, and none of them passes an indicator down to it today.

Opened from	What it knows
Right-click a node in the Health tree (HealthTreeViewNodeLearnThresholdAction)	The learned threshold ids – one indicator, or every indicator of a component
Clicking a threshold curve on the trend chart (IndicatorTrendComposite)	ThresholdLearningData, which holds the threshold ids
The Acknowledge dialog (AcknowledgeHealthNodeComposite)	The same node, so the same ids

All three already have the **learned threshold ids**, so that is what gets passed down rather than an indicator. The check becomes: does any threshold in this relearn have an active deviation. For a single indicator that is exactly the slide's behaviour; for a component-level relearn covering several indicators the warning appears if any of them is deviated, which is the useful answer.

Step 2 — IHealthCommonBusiness and HealthCommonBusiness – a lookup by threshold

mgs-health-business

The client has threshold ids; the deviation is stored against an indicator. The server already knows how to get from one to the other, so the lookup belongs there.

```
// interface
boolean hasActiveDeviationOnThresholds(String aircraftId, List<String> thresholdIds)
    throws EntityNotFoundException;

// implementation
@Override
public boolean hasActiveDeviationOnThresholds(String aircraftId, List<String> thresholdIds)
    throws EntityNotFoundException {
    if (thresholdIds == null || thresholdIds.isEmpty()) {
        return false;
    }
    // The indicators those thresholds belong to
    Set<String> indicatorIds = new HashSet<>();
    HealthMonitoringConfiguration configuration = getLatestConfiguration(aircraftId);
    for (HealthMechanicalComponentConfiguration mcc :
        configuration.getMechanicalComponentConfigurations().values()) {
        for (IndicatorConfiguration ic : mcc.getIndicatorConfigurations().values()) {
            for (ThresholdConfiguration threshold : ic.getThresholdConfigurations()) {
                if (thresholdIds.contains(threshold.getId())) {
                    indicatorIds.add(ic.getIndicator().getId());
                }
            }
        }
    }
    if (indicatorIds.isEmpty()) {
        return false;
    }
    for (IndicatorDeviationDto dto : findActiveDeviations(aircraftId)) {
        if (indicatorIds.contains(dto.getIndicatorId())) {
            return true;
        }
    }
    return false;
}
}
```

Step 3 — DeviationWarningUtils – one more helper

mgs-client/.../presentation/Utils/DeviationWarningUtils.java

Add beside the two already there, so the dialog does not have to handle the exception.

```
/** True when any of these learned thresholds has an active deviation. */
public static boolean hasActiveDeviationOnThresholds(String aircraftId, List<String> thresholdIds) {
    try {
        return business().hasActiveDeviationOnThresholds(aircraftId, thresholdIds);
    } catch (Exception e) {
        LOGGER.warn("Could not check deviations for thresholds", e);
        return false;
    }
}
}
```

Step 4 — LearningPastHoursSelectionComposite – remove the general note

mgs-health-app/.../health/presentation/composite/LearningPastHoursSelectionComposite.java

Delete the whole block added earlier. The warning moves to the composite above, which knows which thresholds are involved; this class is shared and does not.

```
// DELETE
Label deviationNote = new Label(this, SWT.WRAP);
deviationNote.setText(Messages.CORE.getString("deviationRelearnNote"));
GridData noteData = new GridData(SWT.FILL, SWT.BEGINNING, true, false);
noteData.widthHint = 420;
deviationNote.setLayoutData(noteData);
```

Keep the layout change made earlier: `root.setLayoutData(new GridData(SWT.FILL, SWT.TOP, true, false))`. That fixed the radio buttons being covered and is still wanted.

Step 5 — ThresholdLearningComposite – take the threshold ids and show the warning

mgs-health-app/.../health/presentation/composite/ThresholdLearningComposite.java

Two edits.

5.1 Add a parameter to the constructor and keep it:

```

private final List<String> learnedThresholdIds;

public ThresholdLearningComposite(Composite parent, ThresholdLearningModel model,
    long defaultHistoryDuration, List<String> learnedThresholdIds) {
    super(parent, SWT.NONE);
    this.model = model;
    this.learnedThresholdIds = learnedThresholdIds;    // <<< add
    model.addPropertyChangeListener(this);
    createContent(defaultHistoryDuration);
    initializeDataBindings();
    GridLayout layout = new GridLayout(3, false);
    layout.marginWidth = 10;
    setLayout(layout);
}

```

5.2 At the end of createContent, after pastHoursSelectionComposite:

```

// BCR-6605 warn only when one of these thresholds actually has a deviation
if (DeviationWarningUtils.hasActiveDeviationOnThresholds(
    MgsSession.getInstance().getAircraft().getId(), learnedThresholdIds)) {
    Label deviationWarning = new Label(this, SWT.WRAP);
    deviationWarning.setText(Messages.CORE.getString("deviationRelearnIndicatorWarning"));
    deviationWarning.setForeground(getDisplay().getSystemColor(SWT.COLOR_DARK_RED));
    GridData warningData = new GridData(SWT.FILL, SWT.BEGINNING, true, false, 3, 1);
    warningData.widthHint = 480;
    deviationWarning.setLayoutData(warningData);
}

```

Imports: java.util.List, org.eclipse.swt.widgets.Label, com.eurocopter.mgs.presentation.common.MgsSession and com.eurocopter.mgs.presentation.utils.DeviationWarningUtils.

Step 6 — ThresholdLearningDialog – pass the ids through

mgs-health-app/.../health/presentation/dialog/ThresholdLearningDialog.java

The dialog only carries them from the caller to the composite.

```

private final List<String> learnedThresholdIds;

public ThresholdLearningDialog(long defaultHistoryDuration, List<String> learnedThresholdIds) {
    super(Display.getCurrent().getActiveShell());
    this.model = new ThresholdLearningModel();
    this.defaultHistoryDuration = defaultHistoryDuration;
    this.learnedThresholdIds = learnedThresholdIds;    // <<< add
}

@Override
protected Control createDialogArea(Composite parent) {
    parent.setBackground(Display.getDefault().getSystemColor(SWT.COLOR_WHITE));
    parent.setBackgroundMode(SWT.INHERIT_FORCE);
    composite = new ThresholdLearningComposite(parent, model, defaultHistoryDuration,
        learnedThresholdIds);    // <<< add the argument
    composite.pack();
    return composite;
}

```

Step 7 — The two callers – pass what they already have

Both already build the list of threshold ids before opening the dialog, so this is one argument each.

7.1 HealthTreeViewNodeLearnThresholdAction.learnThreshold – the list is built just above:

```

// BEFORE
ThresholdLearningDialog dialog = new ThresholdLearningDialog(maxLearningTime);

// AFTER
ThresholdLearningDialog dialog = new ThresholdLearningDialog(maxLearningTime, learnedThresholdIds);

```

7.2 IndicatorTrendComposite, inside the curve click listener. The ids are on thresholdLearningData, but the dialog is created before they are read – so move that line up:

```

ThresholdLearningData thresholdLearningData = (ThresholdLearningData) chartElement.getData();
List<String> learnedThresholdIds = new ArrayList<>(thresholdLearningData.getThresholdIds());
final ThresholdLearningDialog dialog = new ThresholdLearningDialog(
    thresholdLearningData.getLearningTime(), learnedThresholdIds);
if (dialog.open() == Window.OK) {
    synchronized (this) {
        // the list is already built above, so the old line here is deleted
        String learningReason = dialog.getModel().getLearningReason();
        ...
    }
}

```

AcknowledgeHealthNodeComposite builds its own LearningPastHoursSelectionComposite rather than using this dialog, so it needs no change for the warning. If the client wants the warning there too, the same call can be added to it – computeLearningTime already walks the thresholds and could collect their ids at the same time. Worth asking.

The confirmation popup

Step 8 — OkCreateUpdateListener – Cancel and Continue

mgs-client/.../presentation/aircraft/details/OkCreateUpdateListener.java

Replace the `MessageDialog.openConfirm` block added earlier. `openConfirm` only offers OK and Cancel, so the slide's two buttons need the full constructor.

```
if (healthConfigurationImported
    && DeviationWarningUtils.hasActiveDeviations(aircraftTechnicalId)) {
    MessageDialog dialog = new MessageDialog(aircraftDetailsDialog.getShell(),
        Messages.CORE.getString("deviationConfigUpdateConfirmTitle"), null,
        Messages.CORE.getString("deviationConfigUpdateConfirm"),
        MessageDialog.WARNING,
        new String[] { Messages.CORE.getMnemonicEscapedString("Cancel"),
            Messages.CORE.getString("deviationContinue") },
        1);
    if (dialog.open() != 1) {
        // Cancel, or the window was closed
        return;
    }
}
```

The last argument is the index of the default button, so Continue is focused. `open()` returns the index of the button pressed, and -1 when the window is closed with the cross – so anything other than 1 means do not proceed.

The acknowledge added earlier, after the update succeeds, is unchanged.

Build and test

#	Action
1	Rebuild mgs-health-business, the client and mgs-health-app. Redeploy.
2	Create a deviation on one indicator with a Learned threshold, such as RMSr.
3	Right-click that indicator in the Health tree, Learn threshold – the red warning appears under the radio buttons
4	Do the same on an indicator with no deviation – no warning at all
5	Click a learned threshold curve on the trend chart for the deviated indicator – the warning appears there too
6	Open Update Aircraft for that aircraft – the warning now reads <code>Warning : Deviations are set on this A/C</code>
7	Import a health configuration file and press OK – the popup shows the AirbusWorld instructions with Cancel and Continue
8	Press Cancel – nothing is saved, and <code>end_date</code> in <code>indicator_deviation</code> is still null
9	Press OK then Continue – the update runs and <code>end_date</code> is set

Item 4 is the one that proves the change: before this, the note appeared on every relearn whether or not a deviation existed.